COVTL Lab Console — User Manual

A laboratory bench for articulatory speech synthesis

Frédéric Berthommier

October 2026 — console v3, engine covtl-pipeline v1.1.0

Contents

1 Welcome — what articulatory synthesis is

Most speech synthesizers you have met are *statistical*: they cut recorded audio into pieces and glue them back together. An *articulatory* synthesizer works the other way around. It models the vocal tract itself — the position of the tongue, the lips, the jaw — and computes the sound that such a geometry would produce. You do not record speech; you *build* it, gesture by gesture. The engine behind this console, **covtl-pipeline**, couples the Berthommier COVTL model with the VocalTractLab 2.4 synthesizer.

The COVTL model describes each vocalic gesture as a point in a *polar plane*: a radius ρ (how open or close the gesture is) and an angle θ (where in the vowel space it sits). Consonants live in a second, *consonantal* branch of the same plane. An utterance is a sequence of *targets* — one per phoneme — and the engine interpolates between them. Two properties make this model special:

- **Coarticulation.** A gesture is never finished in isolation; the trajectory toward the next target begins before the current one is fully reached. The curved paths you will see on the polar display are the real engine trajectories, not decorations.
- **Anchors.** The engine plans an utterance as a walk between *anchors* — vowel targets, pauses, the utterance start and end. Everything the console shows you (block boundaries, scrub ticks, spectrogram lines) is derived from that anchor list.

You do not need any programming background to use the console. If you can use a web browser and a media player, you can drive it.

2 Installing

The console is a local web page served by a small Python bridge. Full per-system instructions (Windows, macOS, Linux) live in the repository's `README.md`. **Flash route** (Python 3.9+): `pip install git+https://github.com/FBerthommier/covtl-console.git` then run `covtl-console` — the engine (with its VocalTractLab binaries), the liaison rules and the bridge resolve automatically; on a locked-down machine, clone the repository and run `python install_flash.py`. In short, by hand:

1. Install the engine: clone `https://github.com/FBerthommier/covtl-pipeline` and run `pip install .` inside it (Python 3.9+, 3.12 recommended).
2. Install the cross-word sandhi rules (recommended — the UI preselects the *standard* liaison level): `pip install covtl-sandhi`.
3. In the console folder, run `pip install fastapi uvicorn`.
4. Run `python server.py` and open `http://127.0.0.1:8421` in Chrome, Edge or Firefox.

The header's *backend chip* must read **live**. If it reads *offline*, the bridge is not running yet — start it with `python -m covtl_console` and the console engages automatically — fine for exploring the interface, not for research.

No Python at all? Double-click `covtl-console-standalone.html`: the whole console runs from that single file, and it connects live automatically whenever a bridge is reachable on port 8421.

3 First sounds in five minutes

1. **Pick a language** in the top-left *Language & Speaker* group (say, English). The example chips below the input box reload for that language.
2. **Click the chip** *EN · this is easy for us.* The console previews it immediately: the *Preview stack* on the right fills with three rows — G2P, Enriched, Final SAMPA (Figure 1).

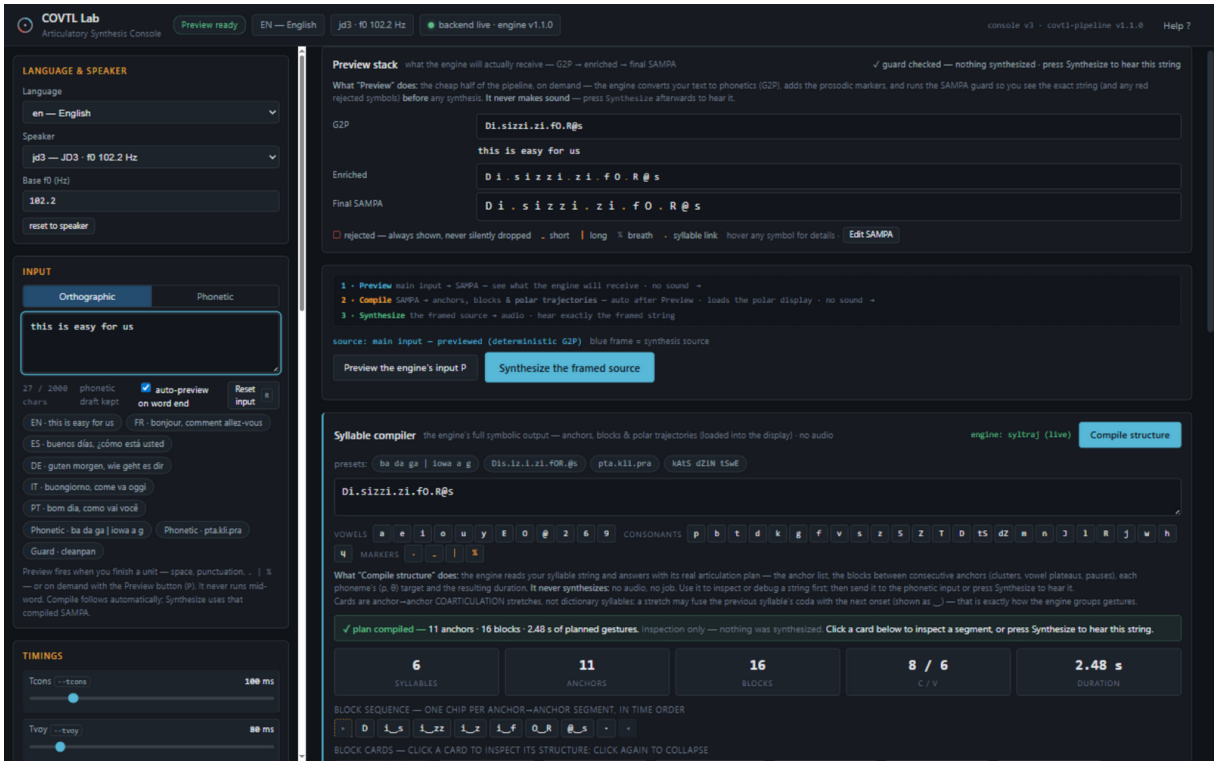


Figure 1: The preview stack on the worked example: G2P output, the enriched string, and the final SAMPA the engine will receive — guard checked, nothing synthesized yet.

3. **Press S** (or click *Synthesize the framed source*). A progress strip runs G2P → trajectory → VTL → WAV; a few seconds later the result summary appears (*2.37 s audio, 11 anchors,* ...).

4. **Press Space** to play. Watch the round *polar display* (Figure 3): a white head travels the trajectory, anchor dots light up, phonetic labels appear at their target positions, and the spectrogram below draws the sound with amber anchor lines.

That is the whole loop: *pick, preview, synthesize, listen, look.*

4 The three-step pipeline

The console is deliberately organized around three questions, printed in the flow guide above the action bar:

1. **Preview — *what will the engine receive?*** Press **P**. The engine converts your text to phonetics (G2P), adds prosodic markers, and runs the SAMPA guard. You see the exact string — including any rejected symbol in red — *before* any synthesis. This step never makes sound.
2. **Compile — *what is the articulation plan?*** The syllable compiler panel runs the engine's own anchor walk over the SAMPA (Section 9). Metric tiles, a chip per segment, clickable block cards. No sound either.
3. **Synthesize — *what does it sound like?*** Press **S**. The engine computes the full trajectory and renders the WAV through VocalTractLab. What you hear is exactly the string you inspected.

The wait indicators. Every step that contacts the engine shows you that it is working: the Preview button relabels to *"Previewing…"* (with its hourglass spinner) and a pill in the header counts *"n in flight…"*. During synthesis, the progress strip lights the current stage (G2P, trajectory, VTL, WAV, MP4) and fills a progress bar. The polar display typically starts moving *while the WAV is still rendering* — that is the start-early feature (Section 7), not a bug.

5 Typing text vs writing SAMPA

The *Input* group has two modes. **Orthographic** is ordinary spelling; the engine's grapheme-to-phoneme module converts it. **Phonetic** is SAMPA, the machine-readable phonetic alphabet the engine speaks; what you type is what the engine receives (after the guard).

Three separators matter, and the engine treats them differently:

- **Dot `.`** — a syllable *link*: no pause, the consonants around the dot form one coarticulated cluster (`ba.da` is one word of two syllables).
- **Space** — a short pause (default 200 ms), written ␣.
- **Vertical bar `|`** — a long pause (default 320 ms), the phrase separator.
- **Percent `%`** — a *breath* pause (130 ms); it also appears as a breath anchor on the display.
- **Tilde `~`** after a vowel — nasality (`a~` is nasalized *a*, as in French *banque*).

The per-language chips (EN, FR, ES, DE, IT, PT, plus two phonetic presets and a guard exercise) fill the input and trigger a preview immediately — the fastest way to hear each language's calibration.

6 The red-reject guard

The engine knows a fixed inventory of symbols (vowels `a e i o u y E O @ 2 6 9` plus nasals, consonants `p b t d k g ...`, and the markers above). Anything else — a stray `c`, an IPA *esh* (`S` in SAMPA), an apostrophe — would be *silently dropped* by the engine's tokenizer. The console refuses to let that happen quietly: such symbols are shown struck through in red, with a count badge and a banner (Figure 2).

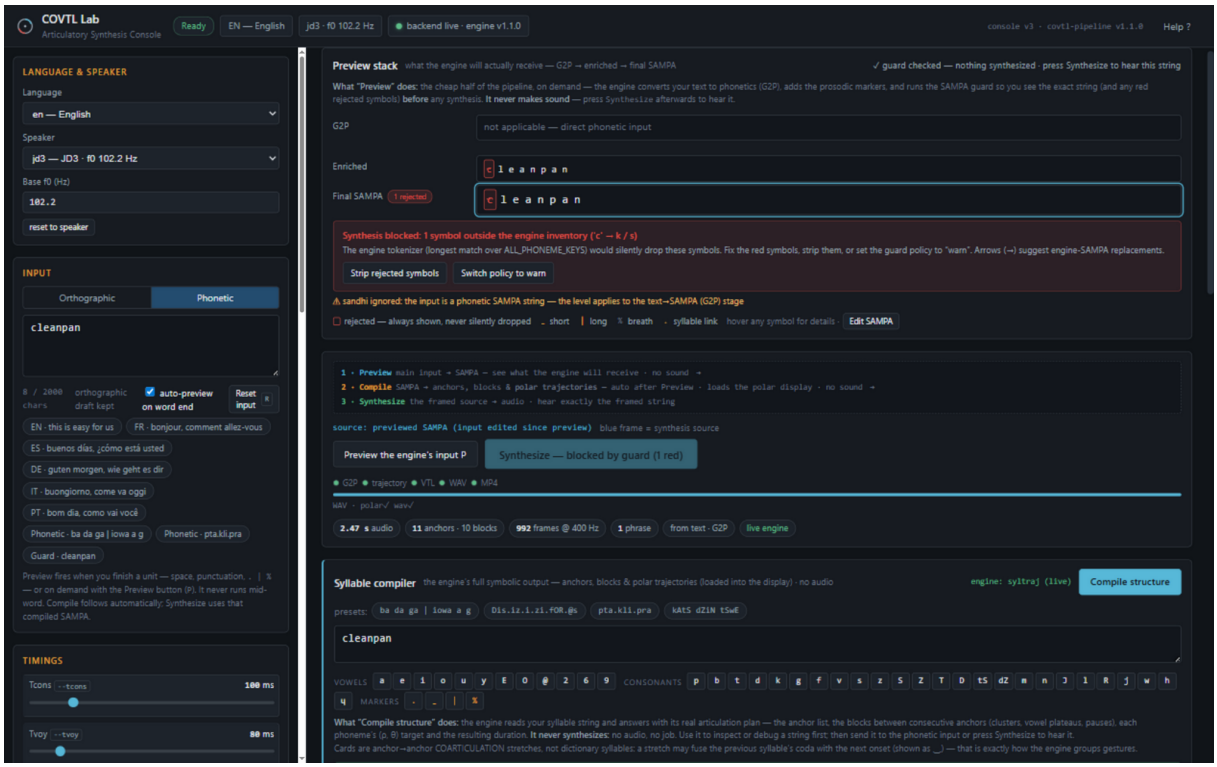


Figure 2: The guard in `error` policy on `cleanpan`: the `c` is rejected with suggestions `k` / `s`, and synthesis is blocked until you fix, strip, or downgrade the policy.

Three policies, in the *Engine & Guard* group:

- **warn** (default) — proceed knowingly: the label changes to *"Synthesize anyway (N dropped)"* and the engine drops the red symbols.
- **error** — blocked: no synthesis until every red symbol is fixed or stripped.
- **off** — pass-through (engine CLI semantics; know what you are doing).

The banner offers one-click *suggestions* (`c` → `k / s`, IPA *esh* → `S`, ...) and a *strip* action that removes all red symbols at once. You can also hand-edit the final string: *Edit SAMPA* opens an editor on the final row; the *edited* badge reminds you the string is yours, and **S** synthesizes exactly that edited string.

7 Reading the continuous polar display

The round display is the heart of the console. Angle θ runs from the top (0°), clockwise; radius ρ is normalized so the engine's own radius range fills the disc (1.0 = the outer dashed ring).

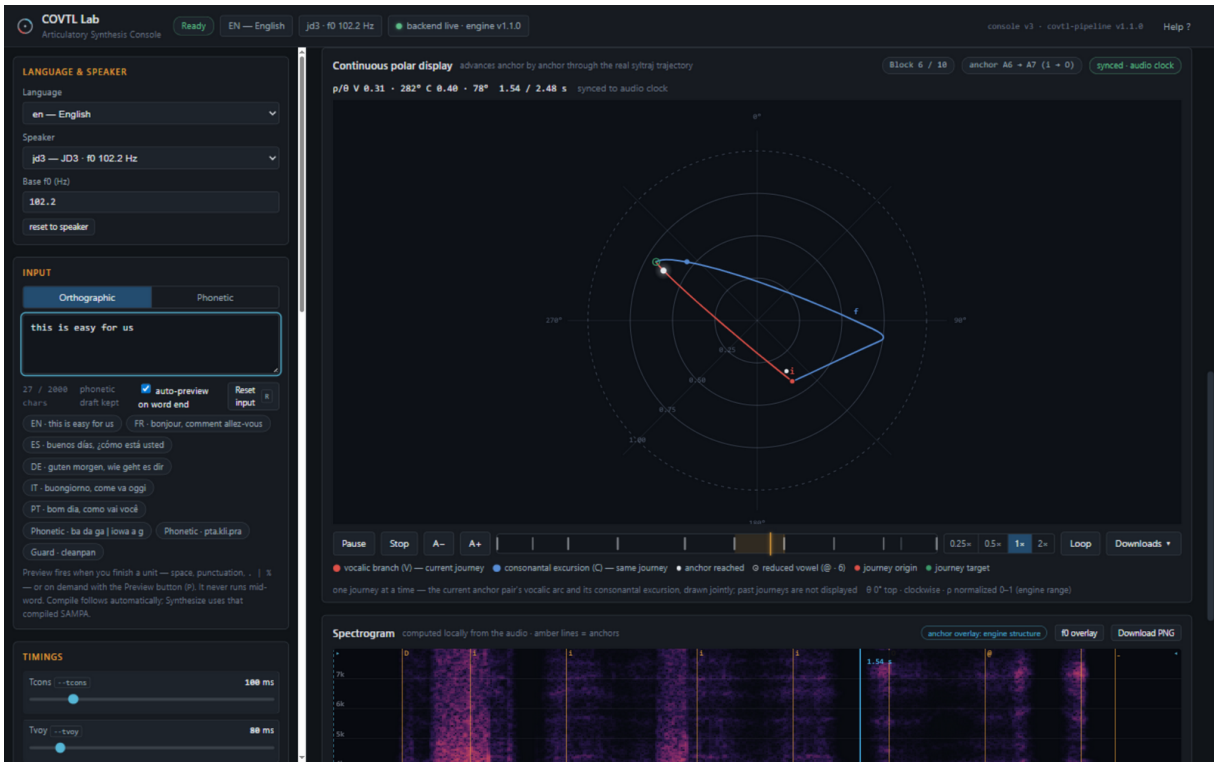


Figure 3: The continuous polar display mid-utterance (Block 5/10, synced on the audio clock). Red trail: vocalic branch; blue dot and ring: consonantal branch; small dots: anchors reached (hollow rings: reduced vowels); red dot: journey ORIGIN, green ring: journey TARGET; labels of the current journey at their phoneme targets.

One journey at a time. The display shows *exactly one journey* — the anchor pair the head is traveling — and eliminates the rest: no watermark of past journeys, the tuilage is a clean switch at the anchor boundary. Within the journey:

- The **vocalic branch** (red) is presented as a *single arc between the start anchor and the end anchor* — the complete gesture, not a stroke chasing the head.
- The **consonantal excursion** (blue) is drawn *jointly* with its vocalic arc whenever the journey carries a cluster: a closed loop attached to the vocalic path at both anchors of the pair. It departs from the vowel being left (or from a silent anchor for an utterance-initial CV), peaks exactly on each consonant target, and lands on the vowel being reached — the teardrop geometry of the big.bi model, $K = 10$. Both elements are always shown together, so the loop reads as one gesture.
- **Reduced vowels** (`@` and `6`) are figured as hollow rings with a center dot — present in the plan, distinct from the filled dots of full vowel targets.
- **Origin vs target.** The journey's two anchors are colored so they can never be confused with the white head: the **origin** (where the gesture leaves) is a **red dot**, the **target** (where the gesture closes) is a **green ring**. On a stationary hold the two stack — the green ring wraps the red dot — and the blue excursion always reads from the red dot to the green ring.

Labels. Phonetic labels appear at their target positions for the current journey only. A count like **"i·4"** means *phonemes share this same target position* within the journey — the engine's

target-sharing, printed as `key·n`.

Stretches. On the compiler cards, a segment label like **"i⁓k"** reads: vowel `i` fused with the following consonant `k`. The tie ⁓ marks consonants that follow a vowel *inside the same anchor-to-anchor stretch* — the coda of one syllable coarticulated with the next onset. That is exactly how the engine groups gestures; it is not a dictionary syllable.

The consonantal loop. The blue excursion is a *closed loop attached to the vocalic path at both anchors of the pair*: it departs from the vowel being left, peaks exactly on each consonant target, and lands on the vowel being reached (the teardrop geometry of the big.bi model, $K = 10$). It is always displayed *together* with its vocalic arc, as one journey.

Start-early. The trajectory (anchors, blocks, polar points) is computed *before* the audio renders. The display therefore starts on its own clock while the WAV is still being computed — honest, because it is the same engine data — and re-anchors seamlessly on the audio clock when the sound is ready. The badge next to the block counter tells you which clock is master: *polar-only* then *synced · audio clock*.

8 The transport

The bar under the polar display is a media player whose *ruler* is the anchor list:

- **Play / Pause** — **Space** or **K**. **Stop** (back to start) — **Shift+S**.
- **[and]** — previous / next *anchor*. The head lands exactly on the anchor, never between.
- **The scrub bar** — its ticks *are* the anchors; click a tick to snap to it. The amber band marks the current block. **Loop** (**L**) repeats the current block.
- **Speeds** — **0**–**3** for 0.25×, 0.5×, 1×, 2×; applies to playback, the polar head and the spectrogram cursor.
- **Downloads** (**D**) — the artifacts of the current job (Section 11).

Clicking a syllable-compiler card after synthesis *seeks* the polar display to exactly that segment — the fastest way to audit one block.

9 The syllable block compiler

Press **C** to focus the compiler (Enter compiles). "Compile" means exactly what the engine does before synthesizing — tokenize the SAMPA, walk the anchors, emit the block schedule — but *without* any synthesis: it is the cheap structural half, so you can iterate freely.

The panel (Figure 4) shows:

- **Metric tiles** — syllables, anchors, blocks, consonant and vowel counts, planned duration.
- **Block sequence** — one chip per anchor-to-anchor segment, in time order; the stretch labels (with the ⁓ marker) appear here.

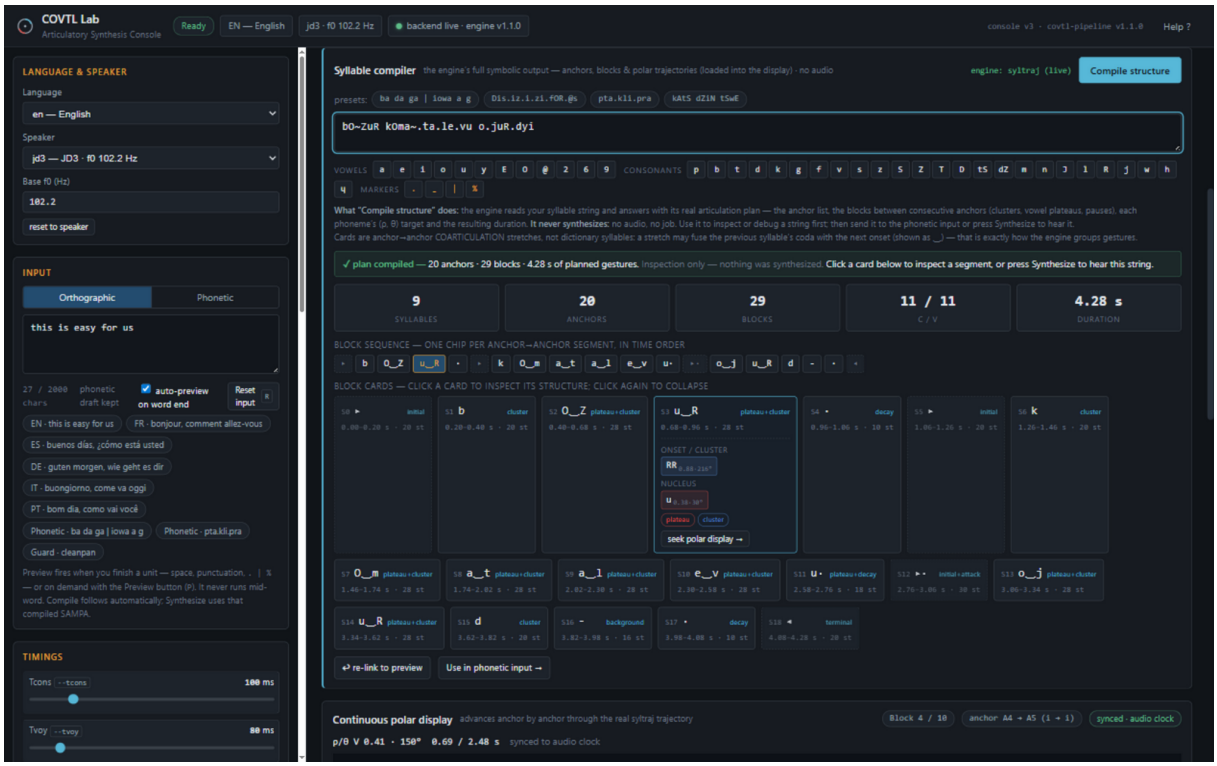


Figure 4: The compiler on the worked example: metric tiles, one chip per anchor-to-anchor segment, and clickable block cards (S3 expanded, showing onset cluster targets in (ρ, θ) and the nucleus plateau).

- **Block cards** — click to expand: onset cluster consonants with their (ρ, θ) targets, the nucleus vowel plateau, block kinds, and time span. *"Use in phonetic input →"* pushes the compiled string to the stage (phonetic mode, previewed); after synthesis, clicking a card seeks the polar display to that segment.

The compiler auto-follows the preview: every preview's final SAMPA is compiled automatically. Type your own blocks to explore — the presets (`ba da ga | iowa a g`, `pta.kli.pra`, ...) are one click away. An *inventory palette* of vowel, consonant and marker chips lets you compose without typing.

10 Timings and expressive prosody

The *Timings* group carries the four engine durations, as steppers (±10 ms):

- **Tcons** — consonantal gesture time (default 100 ms in English).
- **Tvoy** — vocalic plateau time (default 80 ms).
- **tpause** / **tpause-long** — short and long pause durations (200 / 320 ms).

One engine gesture step is 10 ms at the 100 Hz planning rate — durations are rounded to steps. Each language has its own profile; a slider that has drifted from the profile gets an amber edge, and *Reset to language profile* restores it.

The *Prosody* group toggles **expressive prosody**: breathing groups (the `%` breath pause, 130 ms) and an F0 contour — declination over each breathing group, re-attacks, pitch accents on content

words. It touches only the glottis F0 column and pause durations, so it composes with any speaker. The *Base f0* field (60–350 Hz, clamped) is pre-filled from the speaker registry; *reset to speaker* restores it.

The same group exposes **cross-word sandhi** (*liaison*) — `--sandhi off|minimal|standard|careful` and `--sandhi-mode g2p|post|pre`, with **standard preselected by default**. Levels are a monotone coverage filter over each language's rule set (they decide which word-pair classes may fire, never the phonology of a firing rule): *minimal* keeps the categorical, exceptionless rules; *standard* adds the productive ones — French liaison (`faut il` → `fo.til`), English `a apple` → `@.na.p@l`, Italian raddoppiamento, Spanish enlace, Portuguese encadeamento, German Glottalversatz; *careful* is the hyperarticulate register (French *style soutenu*, German boundaries before function words). The level acts at the *text*→SAMPA stage: changing it re-previews, and the compiled plan shows an inserted liaison consonant like any onset cluster (e.g. *comment allez-vous* becomes `kOma .ta.le.vu` with the liaison [t]). The rules live in the companion `covtl-sandhi` package on the engine side — `pip install covtl-sandhi` (see the installation section); the hint under the selects lists the rule modules the bridge actually has. When the package is absent the chosen level degrades to `off` with an explicit warning (bit-identical output).

11 Audio, spectrogram, downloads and exports

When a job finishes, the audio plays through the transport and the **spectrogram** (Figure 5) is computed locally from the WAV. The *amber vertical lines are the anchors* — the same ticks as the scrub bar; click anywhere on the spectrogram to seek, the polar display follows. Buttons: *f0 overlay* (draws the F0 contour on top), *Download PNG*.

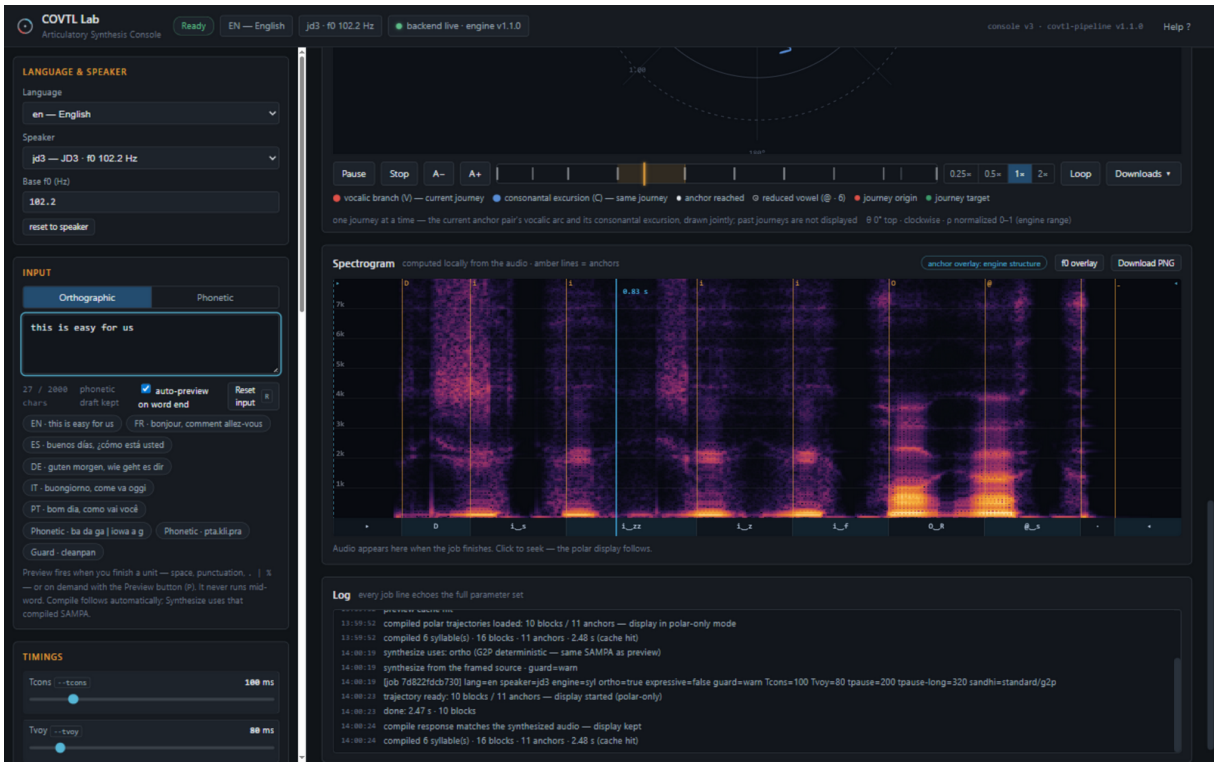


Figure 5: Spectrogram of the worked example with the anchor overlay — click to seek; the polar display follows.

Downloads (D). The WAV and the SAMPA text are *always* offered. The **Optional exports** checkbox (Display & Exports group) adds the TRACT file (the raw articulatory timeline) and the polar data file. Everything lands in the console's `out/` folder; the Display group prints the resolved absolute path so "`out`" is never a mystery.

Video. The *Output* selector can request a **sagittal MP4** (animated mid-sagittal views) or a **dual-panel polar MP4**. Rendering takes markedly longer than audio and the file appears in the Downloads menu when ready; it is not shown inline.

12 Troubleshooting, shortcuts, FAQ

12.1 Troubleshooting

Symptom	Fix
Backend chip reads *demo*	The bridge could not import the engine. Start `python server.py` in the same Python environment you installed covtl-pipeline into; check `/api/status` → `reason`.
Header shows *degraded*	A registry probe failed (speaker or lexicon). Click the backend chip for the full `/api/health` payload; reinstall the language pack or speaker on the engine side.
First preview is slow	The first G2P call loads lexicons (seconds). Later previews are cached and fast.
"Unverified combination" banner	Language-pack vowels are calibrated on the `jd3` speaker only; audio on other speakers may differ. This is a warning, not an error.
No sound	Check the browser's autoplay: click *Play* once (a user gesture unlocks audio). The polar display itself runs regardless.
Synthesis blocked by red symbols	Fix them, use the banner's strip action, or set the guard policy to *warn* (Section 6).
422 on Synthesize	The guard policy is `error` and the input still contains rejected symbols — same fix as above.
Job seems stuck	Long inputs and MP4 modes take minutes. **Esc** cancels.
The two input modes interfere (an old edited SAMPA keeps steering, or a fresh start behaves oddly)	Press **R** (or the *Reset input* button): it clears *both* panes and the whole preview chain — drafts, preview stack, edited SAMPA, guard results — *and* the surfaces built on them: the compiler panel and the synthesis results (playback, polar display, spectrogram, downloads; a running synthesis is cancelled). Your language, speaker, timings and history are kept.
A "·" stretch appears in the plan where a nasal vowel links to the next vowel	A hand-written nasal vowel followed by a linked vowel (`V~.V`, e.g. `kOma .a.le.vu`) is realized by the engine as a *background diphthong arc with a nasalized transition* (V1V2 correction): the plan shows the hiatus as a background stretch labeled with its nasal notation (`a .a`; both members nasal, `V~.V~` → `a .a`) and the audio carries both vowels. The polar display figures the arrival vowel above the green target anchor on such journeys.

The sandhi (liaison) level does nothing, and the preview warns about "covtl-sandhi"	The rule package is not installed on the engine: the chosen level degrades to `off` (bit-identical output). Install `covtl-sandhi` on the engine side (`pip install covtl-sandhi` or copy its `sandhi/` folder over the engine's stub) and restart the bridge; the hint under the Prosody selects lists the rule modules actually available. On a phonetic (SAMPA) input the level is also ignored — it applies to the text→SAMPA stage.
Polar display freezes in a background tab	Expected: browsers throttle background tabs; the clock catches up (watchdog), rendering resumes on focus.

12.2 Keyboard reference

Space/**K**	play / pause	[/]	prev / next anchor
Shift+S	stop (to start)	**L**	loop current block
P	preview (no sound)	**0**–**3**	speed 0.25/0.5/1/2×
S	synthesize the framed source	**R**	reset the input
C	focus compiler	**D**	downloads menu
R	reset the input (panes, chain, compiler, results)	**Esc**	cancel / stop
?	shortcut help		

Shortcuts are plain keys, active when you are *not* typing in an editor.

12.3 FAQ

Is anything sent to the internet? No. The bridge binds 127.0.0.1 only; there is no telemetry and no cloud service.

Why 11 anchors for "this is easy for us" and not 6? The engine's anchor list includes the utterance start, word-end anchors and the terminal anchor — the display follows the engine's own list, by design.

What exactly does Synthesize use? The string shown in the final SAMPA row — whichever source the blue frame shows — the compiler's output when you pushed it to the stage. What you see is what you hear.

Where are my files? In `out/` under the console folder; the Display & Exports group prints the absolute path.

Is there an offline mode? No — and deliberately so. Preview, compile and synthesis are engine calls served by the Python bridge; without it the console stays in an explicit offline state and engages automatically as soon as the bridge answers. Nothing is simulated.